README – Projeto MiniCode
Feito por: Fernanda Matsusita Aguena e Juliana Suchy Panizza

1) **Introdução**
   Este documento tem como objetivo apresentar a arquitetura, os mecanismos de segurança e o fluxo de execução do agente de programação, MiniCode. O projeto foi desenvolvido para a disciplina de Redes Neurais, ministrada pelo professor Edson Takashi Matsubara.

2) **Arquitetura**

   O projeto foi desenvolvido em Python, onde as responsabilidades do agente são divididas entre diferentes componentes. A comunicação com a LLM é realizada por meio da biblioteca openai, utilizando o modelo Qwen/Qwen2.5-14B-Instruct-AWQ e tendo o arquivo llm.py como o responsável por estabelecer essa comunicação e enviar as solicitações ao modelo.

   O arquivo main.py funciona como ponto de entrada do programa, recebendo o diretório do projeto e a tarefa informada pelo usuário. Essas informações são encaminhadas ao agent.py, que atua como orquestrador, coordenando a execução dos demais componentes. A geração da especificação é realizada pelo arquivo spec.py, enquanto o planejamento é produzido pelo plan.py, com ambos utilizando a LLM para gerar as respostas estruturadas que são, posteriormente, validadas e armazenadas em arquivos .md.

   O arquivo states.py implementa a máquina de estados responsável por controlar as transições entre as etapas do agente. Esse componente também estabelece algumas condições que impedem a execução de etapas fora da ordem prevista ou a finalização de tarefas sem a aprovação dos testes, justamente para manter um maior controle sobre o fluxo de execução. A aplicação também conta com ferramentas de leitura e escrita de arquivos e execução de testes, permitindo que o agente realize modificações em projetos de software e verifique os resultados obtidos.

   A interação do agente com os arquivos do projeto é realizada por meio do arquivo tools.py, responsável por disponibilizar ferramentas controladas de leitura, escrita e execução de testes. Essas ferramentas permitem que o MiniCode consulte os arquivos existentes, realize as modificações necessárias para atender à tarefa solicitada e verifique o funcionamento do código produzido. Dessa forma, as operações realizadas pelo agente passam por funções previamente definidas, em vez de permitir que o modelo de linguagem execute comandos livremente no sistema.

   Para garantir maior segurança durante a verificação do código, foi desenvolvido o arquivo executor_isolado.py, responsável por executar os testes em containers Docker. Esse componente prepara uma cópia temporária dos arquivos necessários e realiza a execução em um ambiente isolado, com restrições de acesso à rede, consumo de recursos e permissões. Assim, o código produzido pelo modelo pode

ser testado sem precisar ser executado diretamente no ambiente principal do usuário, reduzindo os riscos associados à execução de código gerado automaticamente.

A verificação dos resultados também é coordenada pelo arquivo verificador.py, que auxilia o agente na identificação de falhas após a implementação. Quando os testes não são aprovados, o resultado da verificação pode ser utilizado pelo agent.py para solicitar uma nova correção ao modelo de linguagem. Esse processo permite que o MiniCode realize tentativas de reparação automática, respeitando o limite estabelecido e utilizando o feedback dos testes para orientar as próximas modificações.

O projeto também conta com o arquivo metricas.py, responsável por calcular informações relacionadas às execuções do agente, como a quantidade de chamadas realizadas ao modelo de linguagem e às ferramentas disponíveis. Essas informações permitem acompanhar o comportamento do MiniCode durante a realização das tarefas e analisar aspectos como a quantidade de verificações e tentativas de reparação necessárias. Além disso, o arquivo relatorio.py é utilizado para organizar os resultados das execuções, contribuindo para o registro e a análise do funcionamento do sistema.

Para avaliar o desempenho do agente em diferentes situações, foi desenvolvido o arquivo cenarios.py, que reúne cinco tarefas de programação envolvendo criação de funções, tratamento de entradas, correção de erros e ampliação de funcionalidades existentes. A preparação dos projetos utilizados nessa avaliação é realizada pelo arquivo preparador_avaliacao.py, que organiza os diretórios e os arquivos necessários para cada cenário, permitindo que as tarefas sejam executadas de maneira independente.

A avaliação dos resultados é realizada por meio do arquivo executar_avaliacao.py, responsável por coordenar a execução dos cenários com o modelo Qwen, acompanhar as tentativas de implementação e reparação e registrar os resultados obtidos. Para evitar que a aprovação dependa apenas dos testes utilizados pelo próprio agente, o projeto também conta com o arquivo avaliacao_independente.py, que executa testes externos de aceitação em um ambiente Docker isolado. Dessa forma, o MiniCode pode utilizar os testes de desenvolvimento para identificar e corrigir problemas, enquanto a avaliação final considera critérios independentes, que não são disponibilizados ao modelo durante a implementação.

Por fim, o arquivo avaliador.py complementa a estrutura de avaliação do projeto, permitindo consolidar os resultados de diferentes execuções e calcular indicadores de desempenho. Entre esses indicadores estão a taxa de sucesso das tarefas, a aprovação na primeira tentativa, a quantidade média de iterações e o número médio de chamadas ao modelo de linguagem. Esses dados possibilitam analisar não apenas se o MiniCode conseguiu concluir as tarefas propostas, mas também como o agente se comportou ao longo do processo de implementação e reparação.

3) Fluxo do agente

O fluxo de execução do MiniCode começa quando o usuário informa o diretório do projeto e descreve a tarefa que deseja realizar. Em seguida, essas informações são recebidas pelo arquivo main.py e encaminhadas ao orquestrador, implementado no agent.py, que coordena as diferentes etapas do agente. Durante a execução dos cenários de avaliação, esse processo também pode ser iniciado pelo arquivo executar_avaliacao.py, responsável por preparar as tarefas e acompanhar os resultados obtidos.

Inicialmente, a tarefa é registrada no estado “RECEIVED” e, após isso, o modelo de linguagem é utilizado para gerar uma especificação contendo o objetivo, os requisitos e os critérios de aceitação da solicitação. Após a validação dessa estrutura, a especificação é armazenada no arquivo specification.md, localizado na subpasta .minicode, e o agente avança para o estado “SPECIFIED”.

Na etapa seguinte, o MiniCode identifica os arquivos existentes no diretório do projeto e encaminha essa informação junto da especificação ao modelo de linguagem. A partir desses dados, é produzido um plano contendo as etapas necessárias para realizar a implementação, os arquivos envolvidos e os testes previstos. O planejamento também passa por uma validação antes de ser armazenado no arquivo plan.md, localizado junto do specification.md, permitindo que o agente avance para o estado “PLANNED”.

Após o planejamento, o fluxo segue para “IMPLEMENTING”, etapa destinada às modificações no projeto. Nessa fase, o modelo de linguagem utiliza as ferramentas disponibilizadas pelo arquivo tools.py para consultar e modificar os arquivos necessários, respeitando as restrições estabelecidas pelo programa. Em seguida, o agente avança para “VERIFYING”, etapa responsável por verificar os resultados da implementação por meio da execução de testes.

A execução dos testes é realizada em contêineres Docker, utilizando o componente executor_isolado.py, que prepara um ambiente separado e aplica restrições de acesso e utilização de recursos. Dessa forma, o código produzido pelo modelo pode ser verificado sem precisar ser executado diretamente no ambiente principal do usuário. Os resultados obtidos são utilizados pelo agente para determinar se a implementação atende aos requisitos estabelecidos.

Caso os testes de desenvolvimento apresentem falhas, o agente poderá entrar no estado “REPAIRING”, realizando novas tentativas de correção até o limite configurado. Durante esse processo, o resultado da verificação é encaminhado ao modelo de linguagem para orientar as modificações necessárias. Após cada tentativa de reparação, os testes são executados novamente, permitindo verificar se os problemas identificados foram corrigidos. Se os testes forem aprovados, a tarefa poderá ser finalizada no estado “DONE”. Caso não seja possível concluir a execução, o agente deverá assumir o estado “FAILED”.

Essas transições são controladas pela máquina de estados implementada em states.py, que estabelece condições para impedir que determinadas etapas sejam

realizadas fora da ordem prevista ou sem atender aos requisitos necessários. Além disso, as informações sobre a tarefa, os resultados das verificações e o histórico dos estados são registrados na subpasta .minicode, incluindo o arquivo state.json, que permite acompanhar o andamento da execução.

Durante a avaliação experimental, o MiniCode também utiliza testes independentes de aceitação, executados pelo componente avaliacao_independente.py. Diferentemente dos testes de desenvolvimento, que fornecem feedback para orientar possíveis reparações, os testes independentes não são disponibilizados ao modelo durante a implementação. Esses testes são utilizados pelo arquivo executar_avaliacao.py para verificar os resultados iniciais e finais de cada cenário, permitindo avaliar o desempenho do agente sem depender exclusivamente das verificações realizadas durante o desenvolvimento.

4) Ferramentas implementadas

As ferramentas implementadas no projeto permitem listar e ler arquivos relevantes, para identificar o contexto de implementação; escrever/atualizar arquivos dentro da área de trabalho autorizada pelo agente; executar testes automatizados com pytest dentro de um container Docker, sem executar o código gerado diretamente no interpretador Python do host. Por ações estruturadas, as ferramentas são chamadas pelo agent.py, cuja integração utilizada não depende de suporte nativo do provedor a tool calling.

5) Spec-Driven Development

A abordagem Spec-Driven Development foi implementada de forma que o agente realize inicialmente a geração de uma especificação e de um planejamento, em vez de produzir diretamente uma solução para a solicitação do usuário. Dessa forma, o projeto busca primeiro definir os requisitos que devem ser atendidos e organizar as etapas necessárias para a implementação, mantendo o controle sobre o andamento do processo por meio das transições da máquina de estados.

A primeira etapa é realizada pelo arquivo spec.py, responsável por encaminhar a solicitação do usuário ao modelo Qwen-14B e gerar uma especificação estruturada em JSON. Essa especificação foi dividida em três campos com nomes autoexplicativos: objective, que descreve o objetivo da tarefa; requirements, que apresenta os requisitos necessários; e acceptance_criteria, que estabelece os critérios de aceitação. Após a geração, a função validar_spec() verifica se os campos obrigatórios estão presentes e possuem a estrutura esperada. Em seguida, o conteúdo é armazenado no arquivo specification.md.

Com a especificação validada, o agente inicia a etapa de planejamento, implementada no arquivo plan.py. Nessa etapa, a especificação e a lista de arquivos existentes no projeto são encaminhadas ao modelo de linguagem, que produz um plano contendo um resumo da implementação (summary), as etapas previstas (steps) e os testes que deverão ser realizados (tests). Assim como ocorre na

especificação, o planejamento também passa por uma validação estrutural, realizada pela função validar_plan(), antes de ser salvo em plan.md.

Para garantir que essas etapas sejam respeitadas, a máquina de estados implementada em states.py impede que o agente avance para IMPLEMENTING sem que a especificação e o planejamento tenham sido validados. Dessa forma, o controle da sequência de execução não depende apenas das instruções fornecidas à LLM, mas também das verificações realizadas pelo próprio programa.

6) Agente hardness

O Agent Hardness foi implementado no projeto por meio de restrições programáticas que controlam as etapas de execução e as ações permitidas ao agente. Essas restrições foram desenvolvidas para evitar que determinadas operações sejam realizadas fora da sequência estabelecida ou sem que as condições necessárias tenham sido atendidas, reduzindo a dependência das decisões tomadas exclusivamente pelo modelo de linguagem. Essas limitações são aplicadas principalmente pelo arquivo states.py, que implementa uma máquina de estados com transições previamente definidas. Dessa forma, o agente não pode avançar livremente entre as etapas, sendo necessário respeitar as condições associadas a cada transição. Por exemplo, a implementação só pode ser iniciada após a validação da especificação e do planejamento, enquanto a finalização da tarefa exige que os testes de desenvolvimento tenham sido executados e aprovados.

Outro mecanismo implementado foi o controle das tentativas de reparação. Caso os testes de desenvolvimento apresentem falhas, a máquina de estados permite a transição para “REPAIRING”, desde que ainda existam tentativas disponíveis. Durante a avaliação experimental, o limite foi configurado para até duas tentativas de reparação por cenário, impedindo que o agente permaneça indefinidamente em um ciclo de correções. Além disso, quando uma nova implementação é iniciada, os resultados dos testes anteriores são invalidados, exigindo que a verificação seja realizada novamente.

As restrições também são aplicadas às ferramentas disponibilizadas pelo arquivo tools.py, que controla as operações de leitura, escrita e execução de testes nos arquivos do projeto. Dessa forma, o modelo de linguagem não recebe acesso irrestrito ao sistema operacional, sendo necessário utilizar as ferramentas previamente definidas para realizar as modificações. A execução dos testes também ocorre de maneira isolada, por meio do componente executor_isolado.py, que utiliza contêineres Docker com limitações de recursos, permissões e acesso à rede. Esses mecanismos contribuem para reduzir os riscos associados à execução de código produzido automaticamente.

Para verificar o funcionamento dessas restrições, foram desenvolvidos testes automatizados com a biblioteca pytest, que avaliam tanto as transições permitidas quanto às situações em que o avanço deve ser bloqueado. Entre os casos testados estão a ausência de especificação ou planejamento válidos, a tentativa de finalizar sem testes aprovados e o controle das tentativas de reparação. Na última execução

registrada da suíte de testes, foram contabilizados 230 testes automatizados, sendo 227 aprovados e 3 ignorados. Esse resultado corresponde à versão do projeto avaliada naquela execução, anterior aos ajustes finais realizados no sistema.

7) Mecanismos de Segurança

O MiniCode considera o código produzido pelo modelo como não confiável e, por isso, adota medidas de segurança que visam separar as atividades de desenvolvimento dos testes de aceitação. Para garantir o isolamento da execução, os testes são realizados em containers Docker com a rede desabilitada, o projeto montado em modo somente leitura e o sistema de arquivos do contêiner protegido contra alterações, permitindo gravações apenas em uma área temporária controlada. Além disso, a execução ocorre com um usuário não privilegiado, com capacidades reduzidas, a opção no-new-privileges habilitada e limites definidos para o consumo de memória, a quantidade de processos e o tempo de execução.

Outra medida adotada consiste na seleção criteriosa dos arquivos utilizados durante os testes, por meio da preparação de uma cópia temporária que aplica filtros para impedir a inclusão de arquivos ocultos e de outros itens inadequados ao ambiente isolado. O MiniCode também mantém os testes independentes de aceitação separados do projeto acessível ao modelo de linguagem (LLM), evitando que o agente tenha acesso direto aos critérios utilizados na avaliação final. Para proteger os testes de desenvolvimento contra possíveis alterações indevidas, o avaliador preserva os critérios estabelecidos com base nos requisitos públicos e restaura os testes de referência antes de realizar a verificação, impedindo que modificações feitas pelo próprio agente determinem o resultado da avaliação. Por fim, a proteção de informações sensíveis é assegurada pelo fornecimento da chave de API por meio de uma variável de ambiente, evitando que esse segredo seja exposto em arquivos do repositório, exemplos de saída ou relatórios.

8) Repair loop

O ciclo de reparação começa quando os testes de desenvolvimento falham. O agente registra a falha, retorna ao modelo o feedback da verificação e solicita uma nova alteração. Em seguida, os testes são executados novamente. O processo termina quando a verificação é aprovada, quando ocorre uma falha de execução ou quando o limite de reparos é alcançado. Há duas camadas distintas de testes: (a) testes de desenvolvimento, conhecidos pelo agente e utilizados para orientar a correção; e (b) testes independentes de aceitação, mantidos pelo avaliador e usados apenas para medir sucesso. Uma aprovação interna não significa, por si só, que a tarefa passou na aceitação externa. Nos cenários C01 (fatorial) e C03 (média), a primeira versão retornava None para entradas inválidas, em lugar de lançar ValueError. Os testes de desenvolvimento detectaram essas falhas; após uma tentativa de reparo em cada cenário, ambas as soluções foram aprovadas pelos testes independentes.

9) Como executar

Pré-requisitos: Python, dependências Python do repositório, Docker Desktop com contêineres Linux e credencial válida para o endpoint Qwen configurado em llm.py. O ambiente utilizado no desenvolvimento foi Windows com PowerShell. Os comandos abaixo devem ser executados na raiz do projeto.

1. Instale as dependências do projeto conforme a lista disponibilizada no repositório e mantenha o Docker Desktop ativo. Construa a imagem de testes:

```
docker build -t minicode-pytest:local ./sandbox
```

2. Configure a chave de API no terminal atual, sem gravá-la no código:

```
$env:MINICODE_API_KEY = Read-Host "Chave do Qwen"
```

Observação: o comando acima exibe os caracteres digitados em versões comuns do PowerShell; prefira um mecanismo de entrada protegida quando necessário. Nunca copie a chave para resultados compartilhados.

3. Execute a avaliação de um cenário com até dois reparos:

```
python executar_avaliacao.py --cenario C01
```

4. Execute os cinco cenários:

```
python executar_avaliacao.py
```

Os resultados são armazenados em uma subpasta de avaliacoes/rodada_..., incluindo resultados.json. Cada projeto pode conter a pasta .minicode/ com estado, artefatos intermediários e histórico. Para publicações, não inclua credenciais nem dados de execução sensíveis.